Design Patterns Course

Learn, Build, Adapt

Session #4

2024

Irina Kamalova

VP Lead Software Engineer @ JPMC
ex-Revolut & ex-Yandex
Built systems to support 45M+ clients

Refreshing: when do we need Creational Patterns?

04.1

Command

Behavioral

Mediator

Visitor

Template Method

Iterator

Command

Memento

Interpreter

Observer

Chain of Responsibility

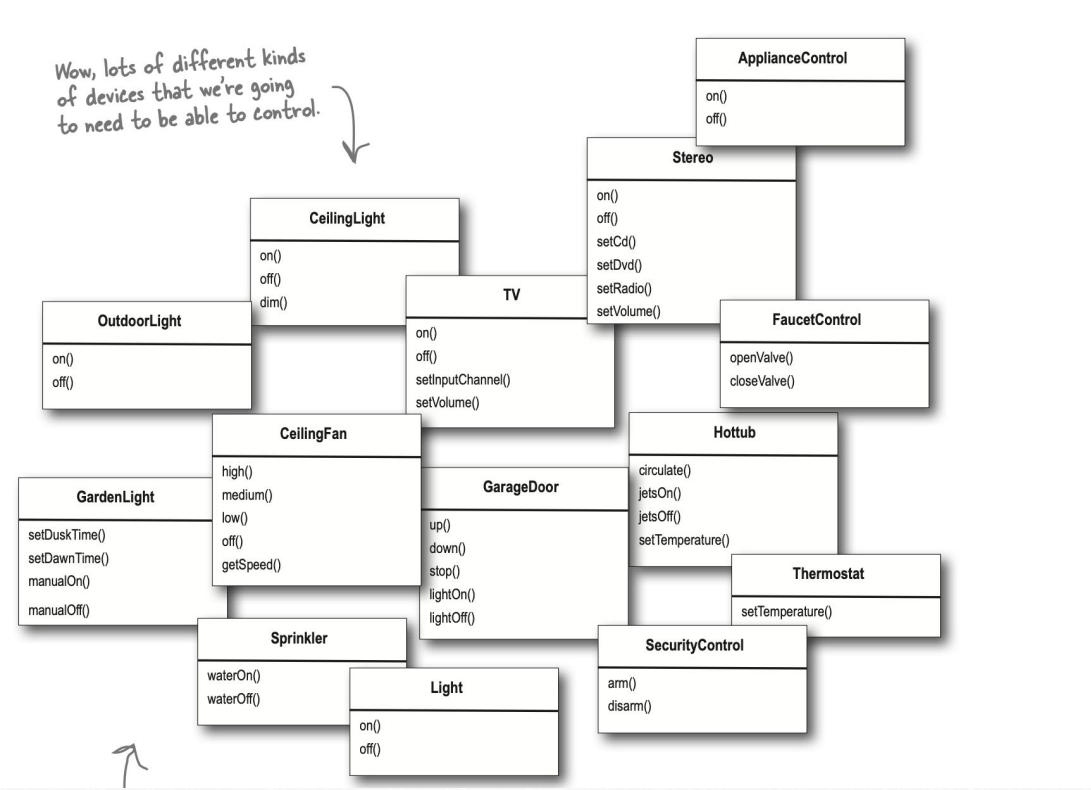
State

Strategy

Behavioral

We **create** a **pattern** for some **behavior** from scratch.

Example


Wow, lots of different kinds of devices that we're going to need to be able to control.
ApplianceControl
on()
off()
Stereo
on()
off()
setCd()
setDvd()
setRadio()
setVolume()
CeilingLight
on()
off()
dim()
TV
on()
off()
setInputChannel()
setVolume()
OutdoorLight
on()
off()
FaucetControl
openValve()
closeValve()
CeilingFan
high()
medium()
low()
off()
getSpeed()
Hottub
circulate()
jetsOn()
jetsOff()
setTemperature()
GardenLight
setDuskTime()
setDawnTime()
manualOn()
manualOff()
GarageDoor
up()
down()
stop()
lightOn()
lightOff()
Thermostat
setTemperature()
Sprinkler
waterOn()
waterOff()
SecurityControl
arm()
disarm()
Light
on()
off()

Command

The Command Pattern encapsulates a request as an object, thereby letting you parameterize other objects with different requests, queue or log requests, and support undoable operations.

Two patterns in one example

Here's the code (the important bits anyway) for the little life-changing application from Chapter 2. See if you can identify who is the Client, who are the Commands, who is the Invoker, and who is the Receiver.

Two patterns in one example

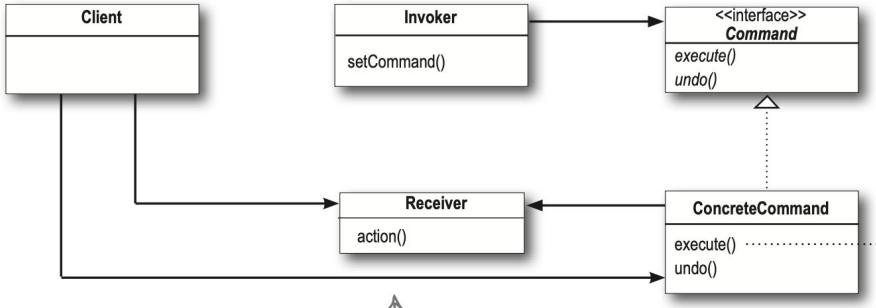
```
public class SwingObserverExample {
       // Set up ...
        JButton button = new JButton("Should I do it?");
        button.addActionListener(new AngelListener());
        button.addActionListener(new DevilListener());
        // Set frame properties here
    }
    class AngelListener implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            System.out.println("Don't do it, you might regret it!");
        }
    }
    class DevilListener implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            System.out.println("Come on, do it!");
        }
    }
}
```

The Command Pattern defined: the class diagram

The Client is responsible for creating a ConcreteCommand and setting its Receiver.

The Invoker holds a command and at some point asks the command to carry out a request by calling its execute() method.

Command declares an interface for all commands. As you already know, a command is invoked through its execute() method, which asks a receiver to perform an action. You'll also notice this interface has an undo() method, which we'll cover a bit later in the chapter.



The execute() method invokes the action(s) on the receiver needed to fulfill the request.

```
public void execute() {
    receiver.action()
}
```

The Receiver knows how to perform the work needed to carry out the request. Any class can act as a Receiver.

The ConcreteCommand defines a binding between an action and a Receiver. The Invoker makes a request by calling execute() and the ConcreteCommand carries it out by calling one or more actions on the Receiver.

Real life example: Slack Bot

Commands:
/start
/help
/remind [time] [date]
/hangout # start a meeting call in this chat
/jira [ticket name] [description] # create a jira ticket from the message
/graphana [cluster] [database] # publish last statuses from graphana metrics on database in provided cluster

Let's consult with Raccoon

https://refactoring.guru/design-patterns/command

Python Example

```python
# Base Command class (Interface for commands)
class ScreenCommand(ABC):
    def __init__(self, screen):
        self.screen = screen
        # Store previous state to support undo functionality
        self.previous_state = screen.text

    @abstractmethod
    def execute(self):
        pass

    @abstractmethod
    def undo(self):
        pass
```

```python
# Receiver class: Screen where the text manipulation happens
class Screen:
    def __init__(self):
        self.text = ""  # Initial screen text content
        self.clipboard = ""  # Clipboard to store cut/copied text

    def paste(self, offset=0):
        # Insert clipboard content at the specified offset
        if self.clipboard:
            self.text = self.text[:offset] + self.clipboard + self.text[offset:]
            print(f"Pasted '{self.clipboard}' at position {offset}. "
                  f"Current text: '{self.text}'")
        else:
            print("Clipboard is empty; nothing to paste.")

    def cut(self, start=0, end=0):
        # Save the cut text to the clipboard and remove it from the text
        if start < end <= len(self.text):
            self.clipboard = self.text[start:end]
            self.text = self.text[:start] + self.text[end:]
            print(f"Cut text '{self.clipboard}' from position {start} to {end}. "
                  f"Current text: '{self.text}'")
        else:
            print("Invalid cut range.")

    def clear_clipboard(self):
        # Clear the clipboard content
        print("Clipboard cleared.")
        self.clipboard = ""
```

Python Example

```python
# Concrete Command to paste text
class PasteCommand(ScreenCommand):
    def __init__(self, screen, offset=0):
        super().__init__(screen)  # Initialize base command with screen
        self.offset = offset  # The position at which text will be pasted

    def execute(self):
        # Perform the paste operation at the specified offset
        self.screen.paste(offset=self.offset)

    def undo(self):
        # Undo the paste by clearing the clipboard and reverting to previous state
        self.screen.clear_clipboard()
        self.screen.text = self.previous_state  # Restore previous state

# Concrete Command to cut text
class CutCommand(ScreenCommand):
    def __init__(self, screen, start=0, end=0):
        super().__init__(screen)  # Initialize base command with screen
        self.start = start  # Starting position for the cut
        self.end = end      # Ending position for the cut

    def execute(self):
        # Perform the cut operation within the specified range
        self.screen.cut(start=self.start, end=self.end)
```

Python Example

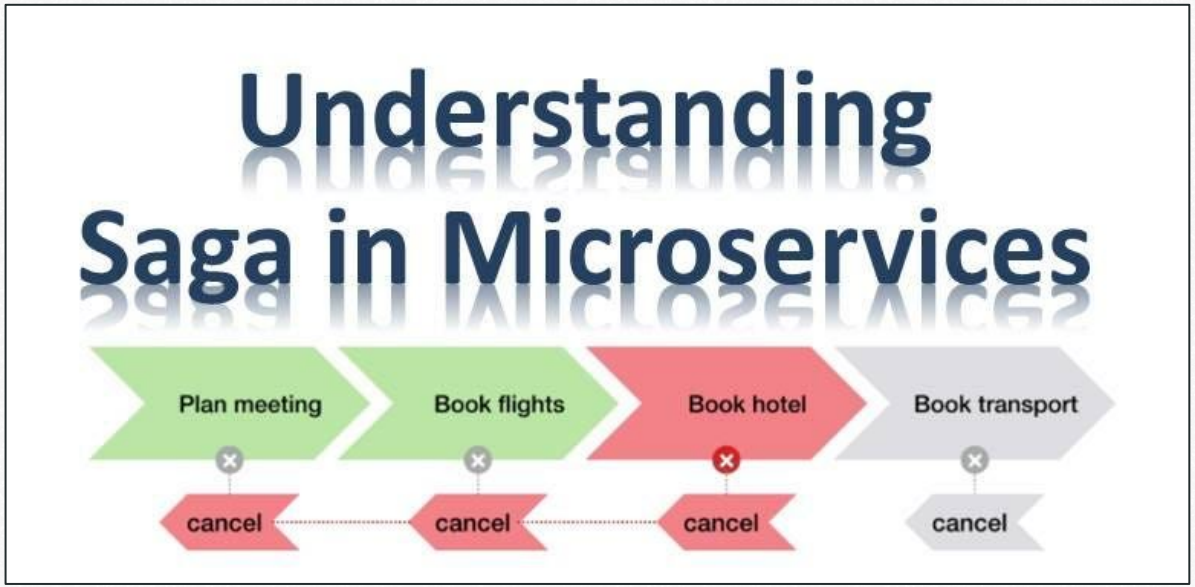
```python
# Invoker class
class ScreenInvoker:
    def __init__(self):
        # Initialize an empty list to keep track of executed commands
        self.history = []

    def store_and_execute(self, command):
        # Execute the command and store it in the history
        command.execute()
        self.history.append(command)

    def undo_last(self):
        # Undo the last command if there is any in the history
        if self.history:
            self.history.pop().undo()
```

```python
# Client Code Example
# Create the Screen (Receiver) and Invoker
screen = Screen()
invoker = ScreenInvoker()

# Execute a series of commands
screen.clipboard = "Hello"  # Simulate storing some text in clipboard
paste_command = PasteCommand(screen, offset=0)
invoker.store_and_execute(paste_command)  # Paste "Hello" at beginning

# Cut text example
screen.text = "Hello World!"  # Set initial text for cutting
cut_command = CutCommand(screen, start=6, end=11)
invoker.store_and_execute(cut_command)  # Cut "World" from text

# Undo the last operation (cut)
invoker.undo_last()  # Should undo the cut, restoring "Hello World!"

# Undo the paste operation
invoker.undo_last()  # Should undo the paste, leaving the text empty
```

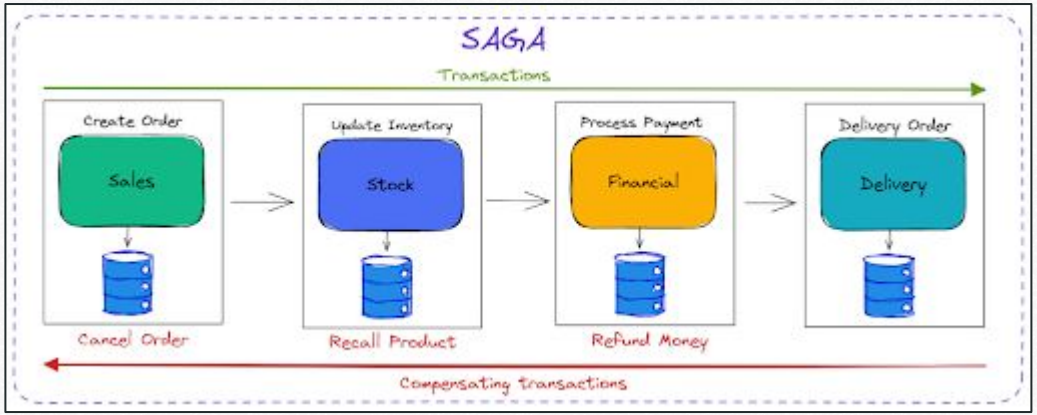
System Design

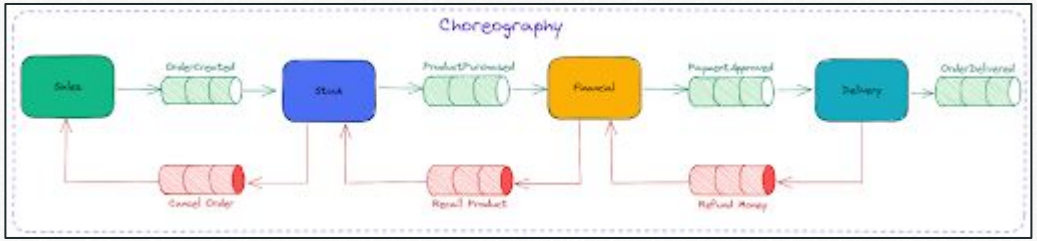
SAGA PATTERN

System Design



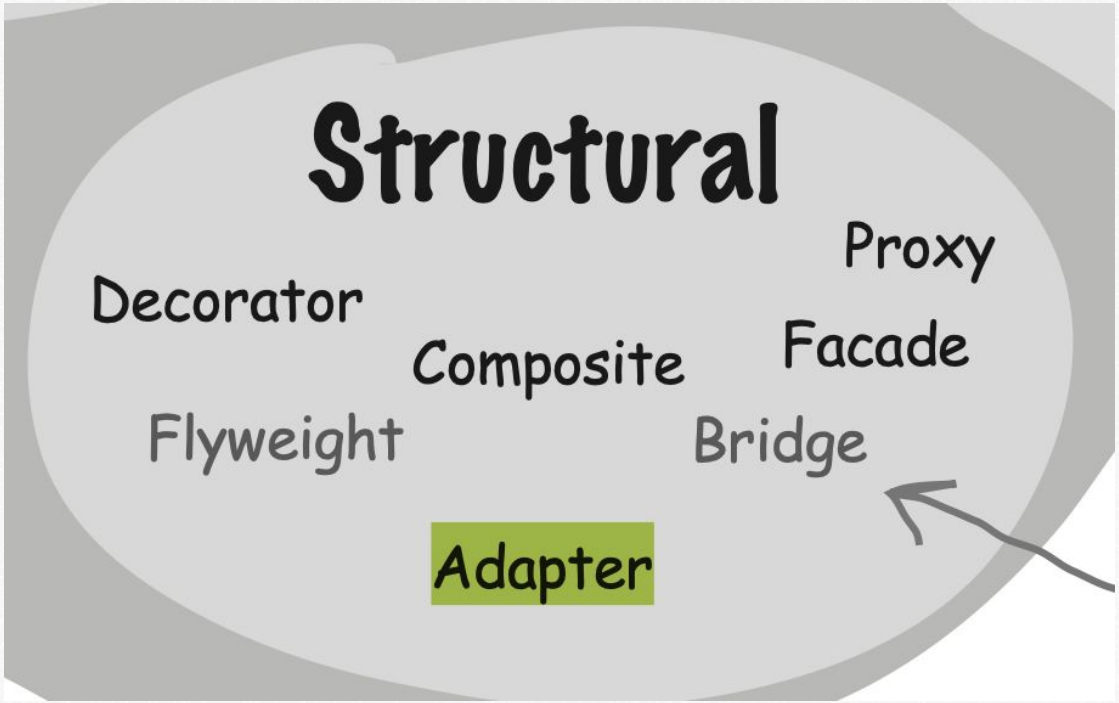
System Design





04.2

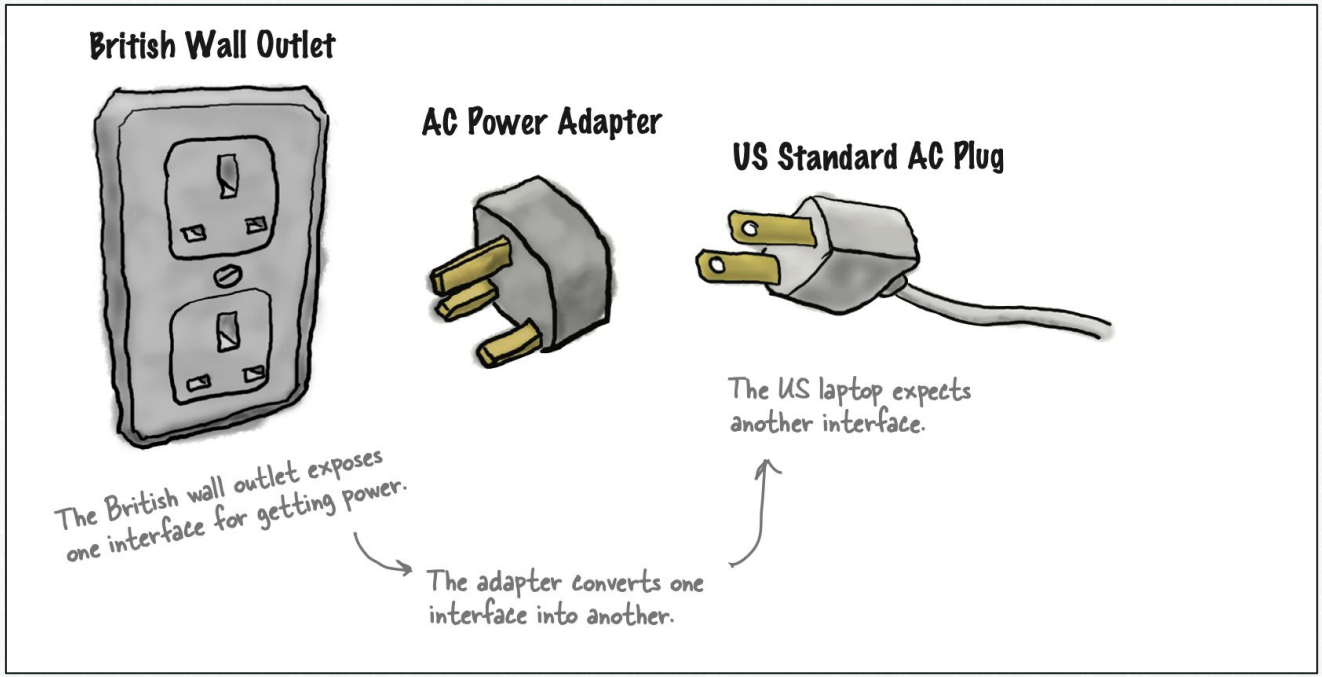
Adapter


Structural
Proxy
Decorator
Facade
Composite
Flyweight
Bridge
Adapter

Structural

We don’t change the behavior of original action/actor/entity, but we **create** a **pattern** for the **structure** to add new behaviour or change the behavior by adding a new layer.

Example


British Wall Outlet
AC Power Adapter
US Standard AC Plug
The US laptop expects another interface.
The British wall outlet exposes one interface for getting power.
The adapter converts one interface into another.

Let's consult with Raccoon

https://refactoring.guru/design-patterns/adapter

Java Code for real life examples

```java
// Target interface for the JSON service
interface JsonService {
    void sendJsonMessage(String jsonMessage);
}

// Adaptee interface for the SOAP service
interface SoapService {
    void sendSoapMessage(String xmlMessage);
}
```

```java
// Adapter class that adapts SOAP to JSON
class SoapToJsonAdapter implements JsonService {
    private SoapService soapService;

    public SoapToJsonAdapter(SoapService soapService) {
        this.soapService = soapService;
    }

    @Override
    public void sendJsonMessage(String jsonMessage) {
        // Convert JSON to XML (this is a simple example, actual conversion will
        String xmlMessage = convertJsonToXml(jsonMessage);

        // Use the adapted SOAP service to send the message
        soapService.sendSoapMessage(xmlMessage);
    }

    private String convertJsonToXml(String jsonMessage) {
        // Simplified conversion logic for demonstration purposes
        // In a real-world scenario, you'd need a proper JSON to XML conversion
        return "<message>" + jsonMessage + "</message>";
    }
}
```

```java
public class AdapterPatternDemo {
    public static void main(String[] args) {
        // Create a SOAP service instance
        SoapService soapService = new SoapServiceImpl();

        // Create an adapter to adapt SOAP to JSON
        JsonService jsonAdapter = new SoapToJsonAdapter(soapService);

        // Example JSON message
        String jsonMessage = "{ \"message\": \"Hello, World!\" }";

        // Use the adapter to send the JSON message
        jsonAdapter.sendJsonMessage(jsonMessage);
    }
}
```

Python Example

```python
# Abstract class for MediaPlayer
class MediaPlayer(ABC):
    @abstractmethod
    def play(self, audio_type: str, file_name: str):
        pass

# Abstract class for AdvancedMediaPlayer
class AdvancedMediaPlayer(ABC):
    @abstractmethod
    def play_vlc(self, file_name: str):
        pass

    @abstractmethod
    def play_mp4(self, file_name: str):
        pass
```

```python
# Implementation of AdvancedMediaPlayer for VLC
class VlcPlayer(AdvancedMediaPlayer):
    def play_vlc(self, file_name: str):
        print(f"Playing vlc file. Name: {file_name}")

    def play_mp4(self, file_name: str):
        # Do nothing
        pass

# Implementation of AdvancedMediaPlayer for MP4
class Mp4Player(AdvancedMediaPlayer):
    def play_vlc(self, file_name: str):
        # Do nothing
        pass

    def play_mp4(self, file_name: str):
        print(f"Playing mp4 file. Name: {file_name}")
```

Python Example

```python
# Adapter class that adapts AdvancedMediaPlayer to MediaPlayer
class MediaAdapter(MediaPlayer):
    def __init__(self, audio_type: str):
        if audio_type.lower() == "vlc":
            self.advanced_music_player = VlcPlayer()
        elif audio_type.lower() == "mp4":
            self.advanced_music_player = Mp4Player()
        else:
            self.advanced_music_player = None

    def play(self, audio_type: str, file_name: str):
        if self.advanced_music_player is None:
            print(f"Invalid media. {audio_type} format not supported")
            return
        if audio_type.lower() == "vlc":
            self.advanced_music_player.play_vlc(file_name)
        elif audio_type.lower() == "mp4":
            self.advanced_music_player.play_mp4(file_name)
```

```python
# AudioPlayer implementation that uses MediaAdapter for non-mp3 formats
class AudioPlayer(MediaPlayer):
    def play(self, audio_type: str, file_name: str):
        # Built-in support to play mp3 files
        if audio_type.lower() == "mp3":
            print(f"Playing mp3 file. Name: {file_name}")
        # Use MediaAdapter for other file formats
        elif audio_type.lower() in ["vlc", "mp4"]:
            media_adapter = MediaAdapter(audio_type)
            media_adapter.play(audio_type, file_name)
        else:
            print(f"Invalid media. {audio_type} format not supported")
```

Python Example

```python
# Demo to show the Adapter pattern in action
if __name__ == "__main__":
    audio_player = AudioPlayer()

    audio_player.play("mp3", "horizon.mp3")
    audio_player.play("mp4", "alone.mp4")
    audio_player.play("vlc", "faraway.vlc")
    audio_player.play("avi", "mindme.avi")
```

```
Playing mp3 file. Name: horizon.mp3
Playing mp4 file. Name: alone.mp4
Playing vlc file. Name: faraway.vlc
Invalid media. avi format not supported
```

Your Journey

01

Introduction
&
Strategy pattern

02

Observer
&
Decorator

03

Creational
Patterns

04

Command
Adapter

05

...

06

...

07

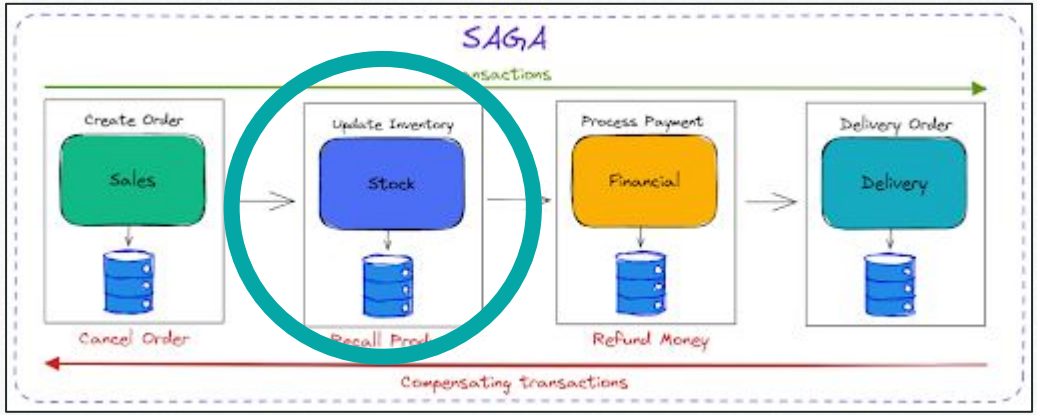
08

...

Combine patterns:

- How system behave? Is there **behavioral** pattern to describe my system?
- Are there **structural** patterns I can change the basic behaviour?
- I defined my behaviour and structure - I use **creational** patterns to create my programme!

Example: Adapter of Order - Stocks



Combine patterns:

- **Behavioral**
  - **Observer** for orders
  - **State** of the Product:
    - ACTIVE (available products)
    - INACTIVE (unavailable products
- **Structural**
  - Decorator for event to be verified
- **Creational**
  - AbstractFactory to build application of Singletons
  - Builder to build the product
  - Prototype to update the product

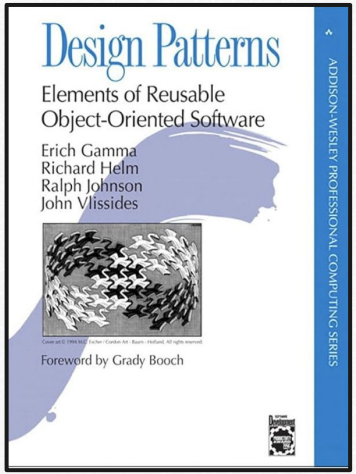
Resources

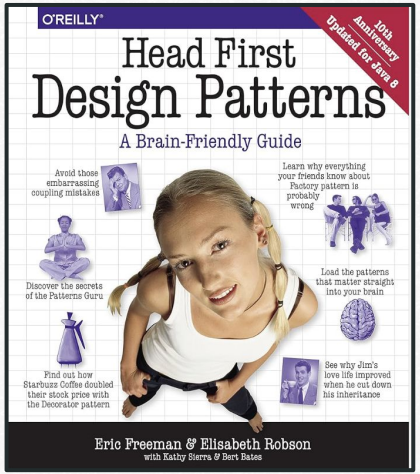
Books:

- Classic GOF Pattern
- Racoon Book
- Head First Design Patterns

Examples:

- Presentation (in Slack)





Thank you!